

# Processamento de OCR em borda e em nuvem aplicado ao *smart metering*

TCC2

Marcelo Fernandes de Oliveira Junior
Gustavo Oliveira dos Santos
Christian Lima Santana

SENAC - Unidade Santo Amaro
Bacharelado em Ciência da Computação
Orientador: Prof. Afonso César Lelis Brandão

São Paulo, 2026

# Roteiro

1. Problema
2. Objetivos e método
3. Desenvolvimento
4. Resultados parciais
5. Andamento

# Introdução - Contexto

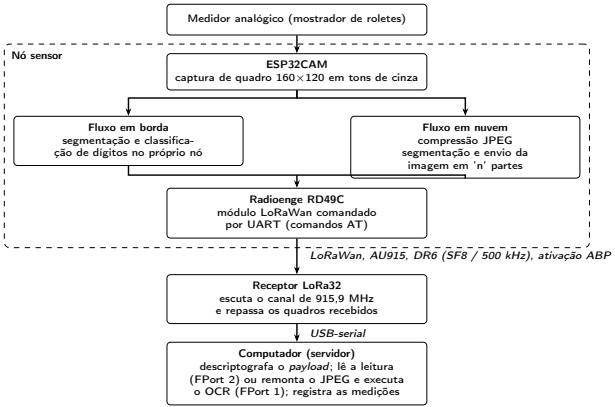
- Medidores analógicos ainda são maioria.
- Trocar por medidores inteligentes é caro.
- ***Retrofitting***: uma câmera lê o mostrador existente.
- Envio por **LoRaWAN**: longo alcance, baixo consumo, pouca banda.



Hidrômetro visto pela câmera do nó (160×120)

# Introdução - Contexto

Como a adoção de determinado método de processamento impacta a eficiência energética, acurácia e desempenho computacional de um sistema IOT voltado à Smart Metering?



# Objetivos

## Objetivo geral

Comparar a viabilidade técnica, a acurácia e o consumo energético do OCR feito **no ESP32-CAM** (borda) e **em um servidor** (nuvem), usando LoRaWAN como meio de transmissão.

### Objetivos específicos

1. Implementar os dois fluxos: OCR no nó e envio da imagem para o servidor.
2. Configurar a rede LoRaWAN buscando o menor consumo possível.
3. Avaliar os fluxos em acurácia (%), latência (s), energia por leitura (mAh) e volume de dados (bytes).

# Hipóteses

## H1

**A quantização em INT8 do modelo da borda reduz a acurácia em relação ao modelo em float32**

## H2

**O processamento local tem custo computacional maior no dispositivo final**

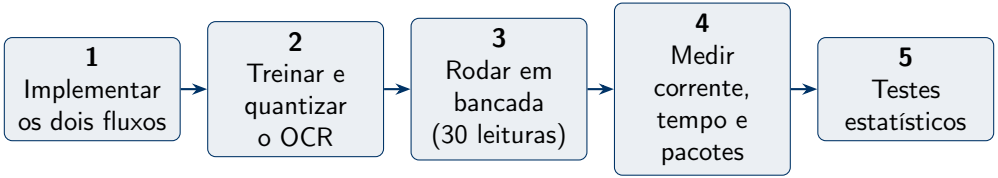
## H3

**A nuvem tem autonomia energética inferior durante a transmissão**

# Como cada hipótese é testada

| | Métrica | Como medimos | Teste |
|---|---|---|---|
| **H1** | Acurácia por dígito e leitura exata | 7.378 dígitos de teste, float32 $\times$ INT8 | McNemar (pareado) |
| **H2** | Tempo de processamento e memória no nó | tempo até a leitura (ou imagem) pronta no nó; uso da PSRAM | comparação direta |
| **H3** | Carga por leitura (mAh) e autonomia | INA219 a 100 amostras/s em 30 ciclos por fluxo | razão entre fluxos e modelo de autonomia |

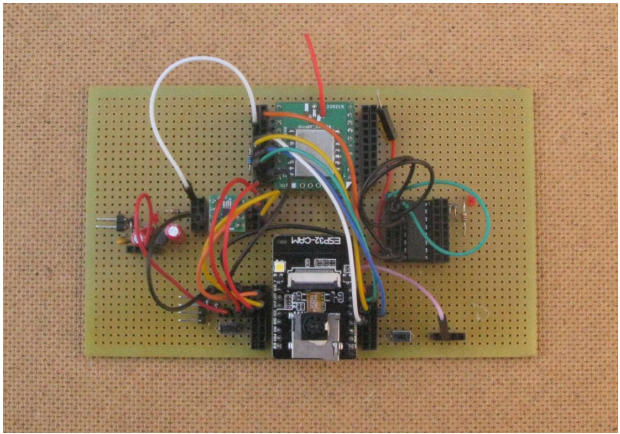
# Método




Pesquisa experimental e quantitativa · mesmo hardware, mesma imagem, mesmo modelo

Dados brutos e *scripts* versionados no repositório do projeto

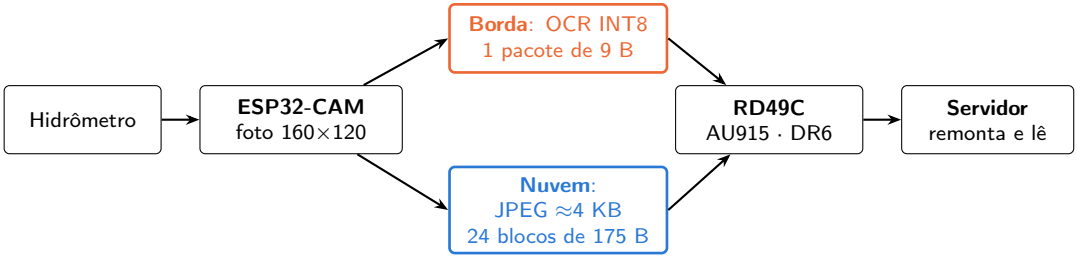
# Hardware




Nó sensor: ESP32-CAM + Radioenge RD49C

- **ESP32-CAM**: câmera e OCR
- **RD49C**: rádio LoRaWAN por comandos AT
- **LoRa32**: receptor no canal 915,9 MHz
- **INA219**: corrente a 100 Hz

# Arquitetura




**SF8 / 500 kHz**

item 10.3 do Ato ANATEL 14.448

**7 s entre blocos**

espera do rádio

**AES + MIC**

ativação ABP

# Modelos Explorados - U-Net

## Métricas e Perda Composta

- **IoU (Intersection over Union)**: mede a sobreposição percentual entre a predição e o gabarito real.
- **BCE + Dice Loss**: a junção da Entropia Cruzada Binária com a *Dice Loss* trata o forte desbalanceamento entre o fundo e a janela do visor.

## Estratégia de Treinamento

- **Duas Fases**: Float32 (até 250 épocas) e QAT (100 épocas).
- **Otimizador Adam**: tx. de aprendizado $10^{-3} \rightarrow 10^{-5}$ na fase QAT.
- **Augmentation**: espelhamento horizontal, brilho, contraste e recortes aleatórios.

Parada antecipada (*early stopping*) aplicada para prevenir sobreajuste (*overfitting*).

# Modelos Explorados — U-Net

## U-Net Principal ($384 \times 384$)

- IoU frequentemente $> 95\%$ em bancada.
- Exigiu $\approx 15$ **MB de RAM** (arena TFLite).

**INVIÁVEL NO ESP32-CAM**

Estoura os $\approx 4$ MB de PSRAM do microcontrolador.

## U-Net Compacta ($160 \times 160$)

- Arena reduzida para **1,7 MB** (viável em RAM).
- Ignorou ruídos visuais (códigos de barras).
- Falhou em **15% dos testes**.

**DESCARTADA (INSTÁVEL)**

Retornava máscara vazia no pós-processamento.

Decisão: substituição da U-Net por rede codificador–decodificador simples.

# Modelos Explorados — CRNN com CTC

## Conceito e Arquitetura

- **CTC Loss**: resolve o alinhamento temporal e reconhece os 8 dígitos sem segmentar a imagem.
- **Convoluções 1D**: substituem LSTMs por *kernels* largos para economizar memória.
- **Entrada**: janela em $48 \times 224$ pixels.

## Estratégia de Treino

- **Dataset Híbrido**: imagens reais recortadas + síntese manuscrita/impressa.
- **Adam + Clipnorm**: $3 \times 10^{-4} \rightarrow 5 \times 10^{-5}$ com `clipnorm=1.0`.
- **Augmentation Agressivo**: ruído gaussiano, brilho e oclusões (blocos pretos/sujeira).

Treinamento durou até 600 épocas em Float32 e 100 épocas na fase QAT.

# Modelos Explorados — CRNN com CTC

## Desempenho em Servidor (30 Imagens)

- **Sequência Completa**: 86,67% (26/30).
- **Dígito (Posição Exata)**: 94,58%.
- **Dígito (Levenshtein)**: 95,42%.
- **Valor Lógico**: 86,67%.

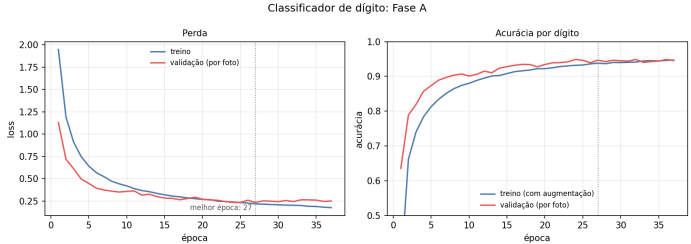
## Inviabilidade no ESP32-CAM

- CTC isolado ocupa apenas $\approx$ 500 **KB**.
- **Gargalo 1**: dependia da U-Net para recortar a janela.
- **Gargalo 2**: ausência do operador `REDUCE_PROD` no TFLite Micro.

**PIPELINE A PRESERVADO**

Mantido como alternativa de alta performance para a nuvem.

# Treinamento




Curvas de treino do classificador de dígitos

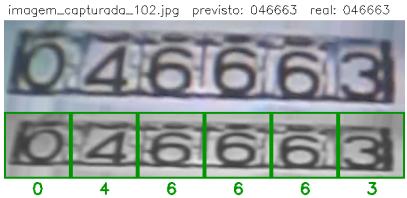
**72 mil**
parâmetros na CNN

**49 mil**
recortes de dígitos no treino

**INT8**
quantização pós-treino

**Dados:** *Word-Wheel Water Meter Dataset*, Zhao et al. (2026), Dryad (≈47 mil recortes) · *Water Meters Dataset*, R. Kucev, Kaggle (localização do visor e ≈1,5 mil recortes) · fotos próprias da ESP32-CAM

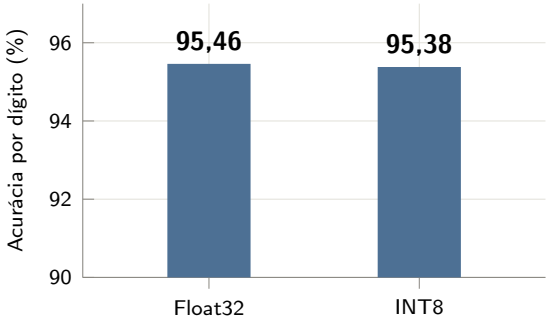
# Leitura no nó




Visor recortado, dígitos e leitura prevista

- Câmera fixa: a janela dos dígitos é **calibrada uma vez**.
- Corta, fatia e classifica cada dígito.
- Roda no ESP32-CAM com TFLite Micro.
- Resultado: uma leitura de **9 bytes**.

# H1 - Quantização INT8

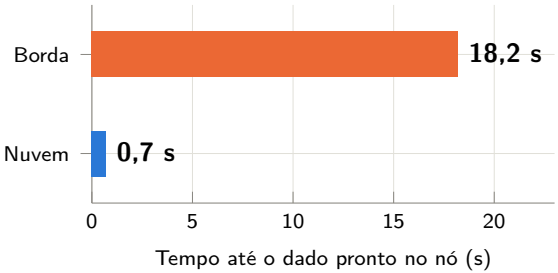



- 7.378 dígitos de teste
- Diferença: **6 dígitos**
- McNemar: $p = 0{,}29$

CONFIRMADA NA DIREÇÃO

mas a perda é desprezível

# H2 - Custo computacional no nó




Borda: captura + localização + 6 dígitos · Nuvem: captura + compressão JPEG

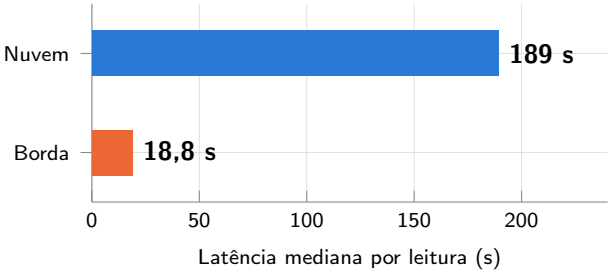
**1,65 MB**
PSRAM usada pelo OCR na borda

**114 mA**
corrente durante a inferência

Mesmo OCR: 17,8 s no ESP32 e 6,25 ms no servidor

CONFIRMADA

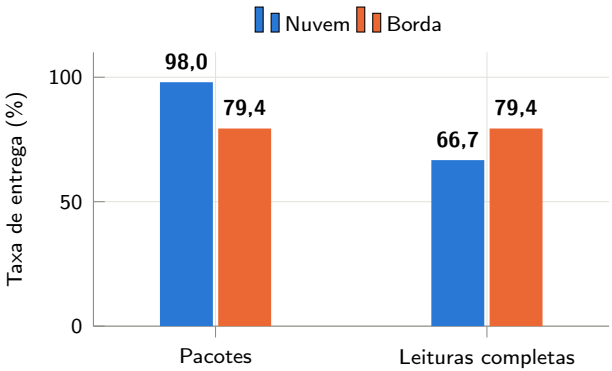
# Rede - Latência




- Nuvem: 24 blocos com 7 s de intervalo
- Borda: inferência ≈18 s + 1 pacote

Borda entrega a leitura **10× mais rápido**

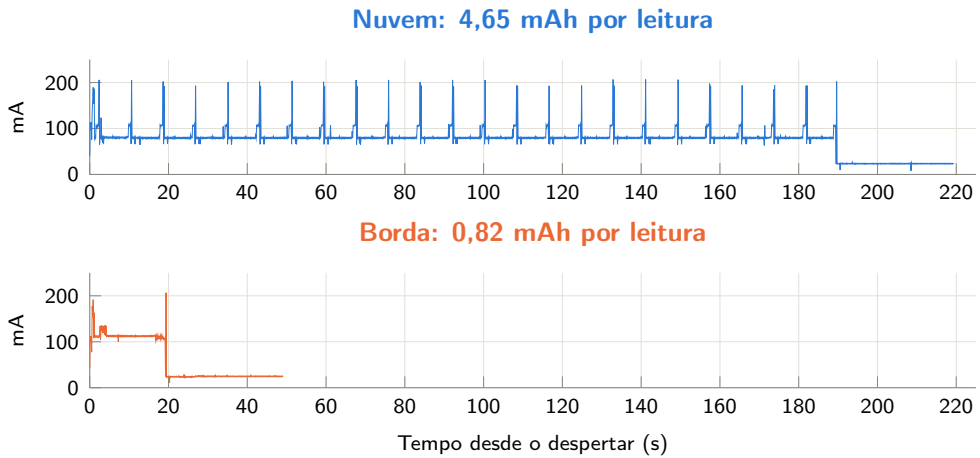
# Rede - Entrega



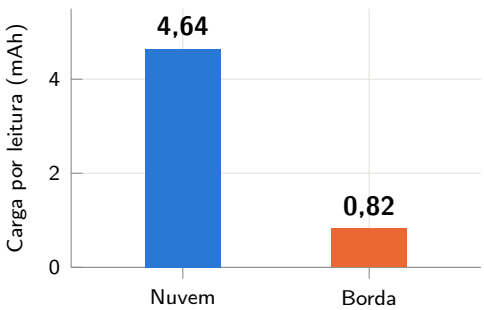

- Nuvem perde **1 bloco** e perde a leitura
- Por leitura: Fisher $p = 0{,}27$

Por leitura, a diferença **não é significativa**

# H3 - Corrente em um ciclo



# H3 - Energia e autonomia




- Nuvem acorda **10× mais tempo**
- Repouso de **24 mA** limita a autonomia
- Ganho de autonomia: **1,0 a 7,0×**, conforme o intervalo

**CONFIRMADA**

# Comparação dos fluxos

| | Nuvem | Borda |
|---|---|---|
| Dados por leitura | ≈4 KB (24 blocos) | 9 B (1 pacote) |
| Latência mediana | 189 s | 18,8 s |
| Carga por leitura | 4,64 mAh | 0,82 mAh |
| Pacotes entregues | 98,0% | 79,4% |
| Leituras completas | 66,7% | 79,4% |
| Inferência | 6,25 ms (PC) | ≈18 s (ESP32) |
| Atualizar o modelo | só no servidor | regravar o nó |

Mesmo modelo INT8 nos dois fluxos

# Síntese das hipóteses

| Hipótese | Evidência | Situação |
|---|---|---|
| H1 - INT8 reduz acurácia | 95,46% $\rightarrow$ 95,38%, $p = 0{,}29$ | DESPREZÍVEL |
| H2 - maior custo no nó (borda) | 18,2 contra 0,7 s; 1,65 MB | CONFIRMADA |
| H3 - nuvem com menor autonomia | 4,64 contra 0,82 mAh | CONFIRMADA |

# Conclusão parcial

- **Borda**: mais rápida e mais econômica.
- **Nuvem**: mais fácil de manter e atualizar.
- Quantização INT8 **não** é o gargalo da acurácia.
- O gargalo é a **captura**: ângulo, luz e fundo do mostrador.




Obrigado!